CMPUT 461/561 Fall 2026

# Lec 04 -- Context-free grammars

© Denilson Barbosa





## Outline

1. Syntax
2. Context-Free Grammars
   - Recursion
   - Counting
3. Approaches for building CFGs
4. BNF
5. Semantic grammars and intent classification

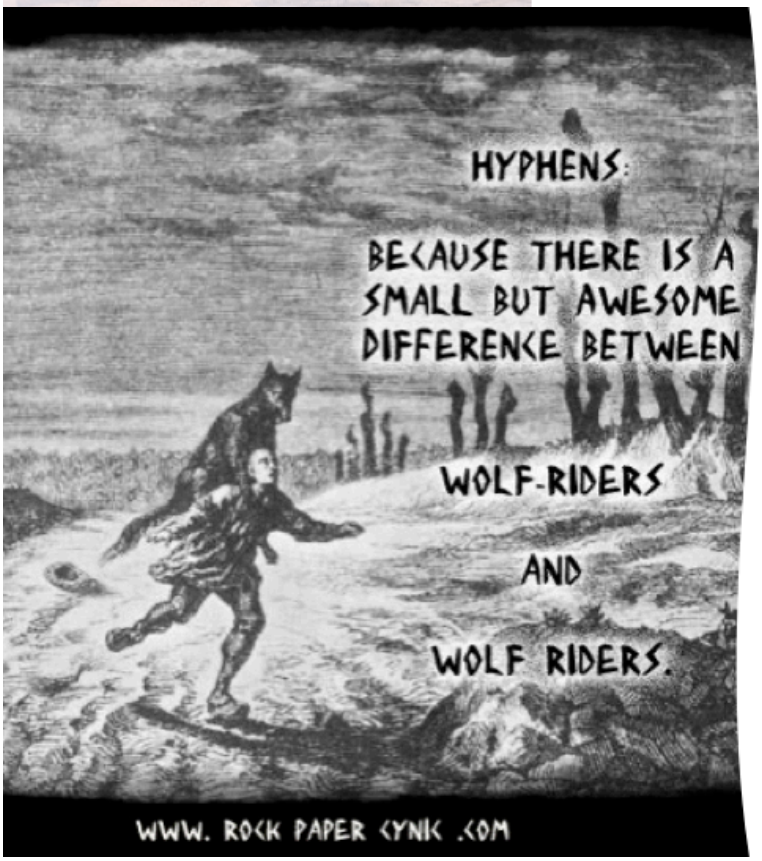


# Syntax

Syntax -- https://en.wikipedia.org/wiki/Syntax

the study of how words and morphemes combine to form larger units such as phrases and sentences






## Why should we care about syntax?

It's important for tools like

- Grammar checkers
- Question answering
- Information extraction
- Machine translation

Syntax is intertwined with meaning

Word order is part of syntax in English

# Syntax of programming languages

The syntax rules of a programming language like Python specify what valid programs look like. https://docs.python.org/3/reference/grammar.html

### Valid

```
if (x > 5):
    print("x is greater than five");
```

### Invalid

```
if (x > 5):
print("x is greater than five");
```

Why is the program invalid?

There is a **rule** in Python that says: code blocks within an `if` statement **must** be indented.

- Omitting the indentation is a **Syntax error**.



## Levels of syntax

Computer language syntax is generally distinguished into three levels. The first two are related to writing only:

- Words – the **lexical level**, determining how characters form tokens;
- Phrases – **the grammar level**, determining how tokens form phrases;

**Lexical error**:

```
NameError
/tmp/ipython-input-3732256527.py in <ce
----> 1 pront("hello world!")

NameError: name 'pront' is not defined
```

**Grammatical errors**:

```
  File "/tmp/ipython-input-868184651.py", line 1
    print("string not terminated)
          ^
SyntaxError: unterminated string literal (detected at line 1)
```

```
  File "/tmp/ipython-input-3922707821.py", line 1
    y = 5 + 22 -
               ^
SyntaxError: invalid syntax
```

https://en.wikipedia.org/wiki/Syntax_(programming_languages)

## Contextual errors in programming languages

The third level of syntax errors in programming languages has to do with **context**:

Context – determining what objects or variables names refer to.

```
NameError
/tmp/ipython-input-816548739.py in
      1 if (5 > 3):
----> 2   print(x)

NameError: name 'x' is not defined
```

Note that a program with only context errors has followed all **writing rules**:

- It only uses correct keywords.
- All expressions are properly formatted.
- ...



# ASIDE: Syntax and semantics

### Syntax has to do with form

Ex: requiring that **strings be enclosed in quotes** says nothing about their meaning.

Syntactically correct code may be meaningless:

```
while(False):
    y = 0
    print(x * y)
    x += 1
```

### Semantics has to do with meaning

Meaning can be defined at different levels: variable, function, module,...

Ex: function that **computes the factorial**:

```
def f(x):
    if (x==0):
        return 1
    else:
        return x * f(x-1)
```

NOTE: often, **we cannot assign meaning** to **syntactically incorrect** code.



## Context-Free Grammars

- Definition
- Example: arithmetic over integers
- Recursion
- Counting
- BNF



## Context-Free Grammars (CFGs)

CFGs are also called **phrase structure grammars**.

CFGs are **declarative**: they describe all the ways in which words in the language can be arranged to form valid phrases.

- Note that FSA are **procedural**: they check (computationally) if a string is in the language.

CFGs have

- **Terminals** – effectively, the words in the language.
- **Non-terminals** – constituents in a language (e.g., noun phrase, verb phrase, sentence).
- **Rules** – equations that have a single non-terminal on the left and any number of terminals and non-terminals on the right.

# Syntax of arithmetic expressions over integers?

What are the rules for writing syntactically valid arithmetic expressions?

| rule | example |
|---|---|
| numbers are valid expressions | 37, 2, 25 |
| we can combine expressions with +, -, *, and / | 37 * 2 / 25 |
| we can have sub-expressions in parentheses | (37 * 2) / 25 |
| we can negate a sub-expression | -5, -(37 * 2) / 25 |



# Syntax of arithmetic expressions: words and phrases

## Words

In the language of arithmetic expressions we have the following "words":

**Operators**: `+`, `-`, `*` and `/`
**Brackets**: `(` and `)`
**Numbers**:



## Phrases/sentences

Rules for combining "words" to form syntactically valid "phrases" (expreesions):

A **number** is an **expression**.

If $E_1$ and $E_2$ are **expressions**, so are:
`(` $E_1$ `)`, $E_1$ `+` $E_2$, $E_1$ `-` $E_2$
$E_1$ `*` $E_2$, $E_1$ `/` $E_2$ and
`-` $E_1$.

## Grammar for arithmetic expressions

E → NUM
E → NEG E
E → LPAR E RPAR
E → E PLUS E
E → E MINUS E
E → E TIMES E
E → E DIV E

NUM → $x \in \mathbb{Z}$ NEG → `-`
LPAR → `(` RPAR → `)`
PLUS → `+` MINUS → `-`
TIMES → `*` DIV → `/`

### CFG Terminology

**Terminals** are the words.
Ex: `(`, `+`, or a number.

**Non-terminals** correspond to constituents.
Ex: E, NUM, LPAR, ...

**Rules**: A → (RHS)
Ex: E → E MINUS E
(meaning: the difference between two expressions is also an expression).



# ASIDE: CFGs, PoS taggers, ...

CFGs were developed independently of PoS tagging systems, and there is some overlap.

For example, while not required, most CFGs use PoS tags as non-terminals. Also, for simple languages (like the ones in textbook examples), the CFG contains productions associating words (terminals) to the "tags".

**Example**:

```
S -> NP
NP -> DT NN .
DT -> 'The' | 'A' | 'the' | 'a'
NN -> 'cat' | 'dog' | 'koala' | 'whale'
. -> '.'
```

In a production system, often, we can perform PoS tagging separately from the grammar and attempt to parse the text using (all combination of) the PoS tags assigned to the words by an independent tagger.

# ASIDE: CFGs, PoS taggers, ...

The (very practical) issue of which system performs tagging is apparent in our example grammar for arithmetic expressions.

Note that the **literals** (i.e., the "words") this language are numbers, operators, and parenthesis.

We can list all operators in the language, as there is a finite number of them.

We cannot list all numbers, though, so we need a system to "tag" them on behalf of the "grammar processor".

NUM → $x \in \mathbb{Z}$
NEG → `-`
LPAR → `(` RPAR → `)`
PLUS → `+` MINUS → `-`
TIMES → `*` DIV → `/`



## The NLTK grammar package

**Grammar**:

```
grammar = CFG.fromstring("""
   E -> NUM
   E -> LPAR E RPAR
   E -> NEG E
   E -> E PLUS E
   E -> E MINUS E
   E -> E TIMES E
   E -> E DIV E
   NUM -> '#NUM#'
   LPAR -> '('
   RPAR -> ')'
   NEG -> '-'
   PLUS -> '+'
   MINUS -> '-'
   TIMES -> '*'
   DIV -> '/'
""")
```

**Using the grammar**

```
sent = '( 37 * 2 ) / 5'.split()
sent = ['#NUM#' if i.isdigit() else i for i in sent]

parser = ChartParser(grammar)
for tree in parser.parse(sent):
    print(tree)
```

Note the Python "trick" to convert arbitrary numbers into the `#NUM#` "tag":
Again, in a production system, we would have a proper tagger for the whole text.

# Formal definition of a CFG

A Context-Free Grammar (CFG) is a tuple $(N, \Sigma, R, \sigma)$ where:

- $N$ is a set of **non-terminal** symbols (also called **variables**).
- $\Sigma$ is a set of **terminal symbols** (disjoint from $N$ and sometimes called the *lexicon*).
- $R$ is a set of **rules** (also called **productions**), each of the form $A \rightarrow \beta$,
  where $A$ is a non-terminal, and
  $\beta$ is a string of symbols from (the infinite set) $(\Sigma \cup N)*$
- $\sigma \in N$ is a designated **start symbol** (normally **S**).

NOTE: most systems, including NLTK, do not require an explicit start symbol.

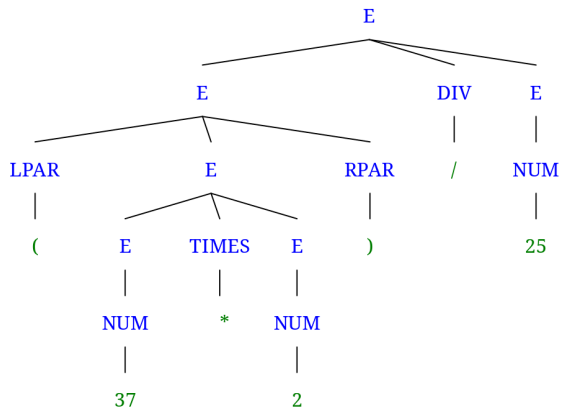


## Grammars as sentence "generators"

If a sentence is grammatical, we can "generate" it from the **start symbol** in the grammar, by applying **rules**, forming a tree where leaves are **terminals**.

The tree to the right is the "generation" of the grammatical expression

```
(37 * 2) / 25
```



# Recursion

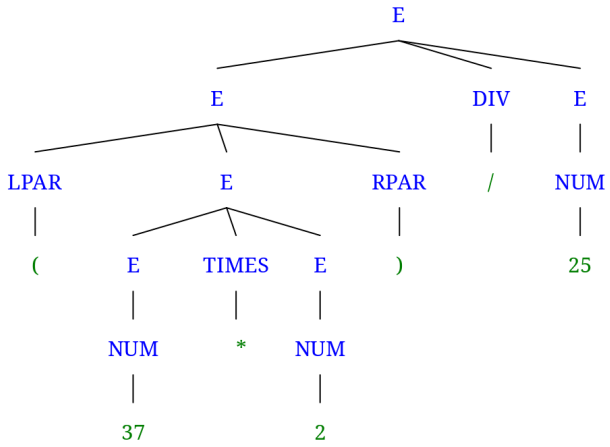
Unlike regular expressions and FSA, CFGs capture recursive phenomena. Note that expressions are recursively defined in the grammar:

"The negation of an expression is also an expression"

$$E \rightarrow NEG\ E$$

"An expression enclosed in parentheses is also an expression"

$$E \rightarrow LPAR\ E\ RPAR$$



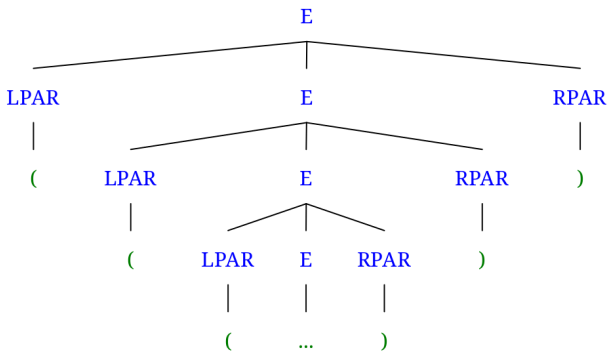



# Counting

CFGs can count! Note that the productions below ensure that in every grammatical expressions, **every `(` is matched by a `)`**!

$$E \rightarrow LPAR\ E\ RPAR$$
$$LPAR \rightarrow ($$
$$RPAR \rightarrow )$$

Therefore, the number of `(` must be the same as the number of `)`, whatever the number may be.





# Building CFGs



## Constituency

> In syntactic analysis, a **constituent** is a word or a group of words that function as a **single unit** within a hierarchical structure.
>
> https://en.wikipedia.org/wiki/Constituent_(linguistics)

Typically, constituents are **phrases**.

- A phrase is a sequence of one or more words built around a **head lexical item** and working as a unit within a sentence.

While no crisp algorithm exists, we find constituents by applying some tests.

## Some constituency tests for English

Example sentence: `Drunks could put off the customers.`

### Coordination test

**Assumption**: only constituents can be joined (with `and`, `or`, ...).

✅ `[Drunks and bums] could put off the customers` → `Drunks` is a constituent!

✅ `Drunks could [put off or scare] the customers` → `put off` is a constituent!

❌ `Drunks could [put or scare] off the customers` → `put` is **not** a constituent!



## Some constituency tests for English

Example sentence: `Drunks could put off the customers.`

### Proform substitution

**Assumption**: only constituents can be replaced by *proforms* (pronoun, pro-verb, ...)

✅ `[They] could put off the customers` → `Drunks` is a constituent!

✅ `Drunks could [do it to] the customers` → `put off` is a constituent!

❌ `[???] put off the customers` → `Drunks could` is **not** a constituent!

## Some constituency tests for English

Example sentence: `Drunks could put off the customers.`

### Answer fragments test

**Assumption**: only constituents can answer grammatical wh-word (*who*, *what*, ...) questions

✅ `[What] could drunks do?` → `put off the customers` is a constituent!

✅ `[Who] could drunks put off?` → `the customers` is a constituent!

❌ `What could drunks do about putting the customers?` → `off` is **not** a constituent!



# Constituency and grammars

It isn't easy or obvious how we come up with the right set of constituents and rules for how they can be combined.

There are many different theories of grammar and competing analyses of the same data.

Nevertheless, **productions in a good grammar** should keep typical constituents together!

| | |
|---|---|
| the/DT customers/NNS | NP → DT NNS |
| eat/VB the/DT fruit/NN | VP → VB NP  NP → DT NN |
| put/VB off/RP the/DT customers/NNS | VP → VB RP NP |

## Guidelines for building CFGs

Expect to work iteratively. Organize yourself.

- Build a "development set" of sentences that you use to guide your work.

Look at many (PoS-tagged) sentences before you start.

- Avoid writing a separate rule for each sentence -- that leads to madness.

Work on finding constituents -- that really helps with finding the syntactical "building blocks" of the language.

Group similar sentences together.

- For example, look at several "Subject-Verb-Object" (ex: `I ate the pie`) sentences together; try to write rules for them; then work on more complex sentences next.





# Backus-Naur Form

# Backus-Naur Form (a.k.a., Backus Normal Form)

> In computer science, **Backus–Naur form** (BNF) is a notation system for defining the syntax of programming languages and other formal languages, developed by John Backus and Peter Naur.
> https://en.wikipedia.org/wiki/Backus–Naur_form

Like CFGs, the BNF notation also consists of variables, terminals and rules.

- Rules look like `Non-terminal ::= expression`
- Expressions mix variables and terminals; and multiple rules are combined with `|`



## Example BNF notation

S → NP VP

NP → Det Nominal

Nominal → Noun

VP → Verb

VP → Verb Adv

Det → a | the | some

Noun → flight | bird | book

Verb → leave | like | read

Adv → slowly | happily

```
<S> ::= <NP> <VP>
<NP> ::= <Det> <Nominal>
<Nominal> ::= <Noun>
<VP> ::= <Verb> | <Verb> <Adv>
```

```
<Det> ::= 'a' | 'the' | 'some'
<Noun> ::= 'flight' | 'bird' | 'book'
<Verb> ::= 'leave' | 'like' | 'read'
<Adv> ::= 'slowly' | 'happily'
```

NOTE: typically, with BNF, the list of words for each variable is provided separately from the main grammar (file).

# Mixing terminals and non-terminals

Although "pure" grammar notation would have a separate production for each of the valid sentences of the same type, most systems are not so strict.

```
grammar = CFG.fromstring("""
   E -> NUM
   E -> LPAR E RPAR
   E -> NEG E
   E -> E PLUS E
   E -> E MINUS E
   E -> E TIMES E
   E -> E DIV E
   NUM -> '#NUM#'
   LPAR -> '('
   RPAR -> ')'
   NEG -> '-'
   PLUS -> '+'
   MINUS -> '-'
   TIMES -> '*'
   DIV -> '/'
""")
```

```
grammar = CFG.fromstring("""
   E -> NUM | '(' E ')' | '-' E | E OP E

   NUM -> '#NUM#'
   OP -> '+' | '-' | '*' | '/'
""")
```

Both grammars work with the same sentences.

- Although their parse trees are **very** different.



## Example

Which sentences are generated by this grammar?

1. a morning like
2. it is non-stop
3. I need a flight
4. that flight is from Boston

| | | |
|---|---|---|
| *Noun* | → | *flights* \| *flight* \| *breeze* \| *trip* \| *morning* |
| *Verb* | → | *is* \| *prefer* \| *like* \| *need* \| *want* \| *fly* \| *do* |
| *Adjective* | → | *cheapest* \| *non-stop* \| *first* \| *latest* \| *other* \| *direct* |
| *Pronoun* | → | *me* \| *I* \| *you* \| *it* |
| *Proper-Noun* | → | *Alaska* \| *Baltimore* \| *Los Angeles* \| *Chicago* \| *United* \| *American* |
| *Determiner* | → | *the* \| *a* \| *an* \| *this* \| *these* \| *that* |
| *Preposition* | → | *from* \| *to* \| *on* \| *near* \| *in* |
| *Conjunction* | → | *and* \| *or* \| *but* |

**Figure D.2** The lexicon for $\mathscr{L}_0$.

| Grammar Rules | | | Examples |
|---|---|---|---|
| *S* | → | *NP VP* | I + want a morning flight |
| *NP* | → | *Pronoun* | I |
| | \| | *Proper-Noun* | Los Angeles |
| | \| | *Det Nominal* | a + flight |
| *Nominal* | → | *Nominal Noun* | morning + flight |
| | \| | *Noun* | flights |
| *VP* | → | *Verb* | do |
| | \| | *Verb NP* | want + a flight |
| | \| | *Verb NP PP* | leave + Boston + in the morning |
| | \| | *Verb PP* | leaving + on Thursday |
| *PP* | → | *Preposition NP* | from + Los Angeles |

**Figure D.3** The grammar for $\mathscr{L}_0$, with example phrases for each rule.



# "Semantic" grammars and intent classification



## Travel agent assistant

General-purpose grammar:

S → VP | NP VP  VP → VB NP

NP → DT NP | NN PP | PRP | TO VP

PP → IN NNP | IN NN

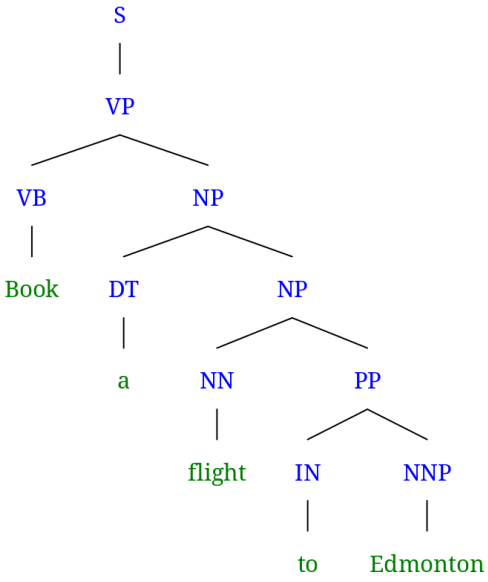
VB → book | cancel | find | want

DT → a | the | this | that

NN → book | flight | reservation | trip

NNP → Edmonton | Calgary | Suzy | Rio

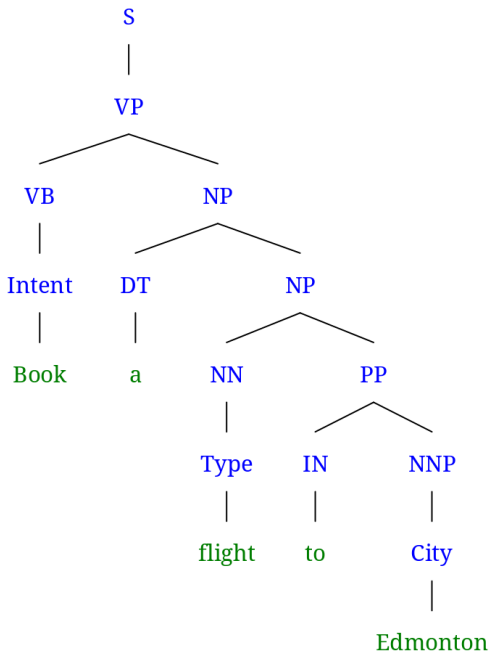
IN → with | to | in | on  TO → to



## Travel agent assistant

Specialized "semantic" grammar:

S → VP | NP VP  VP → VB NP
NP → DT NP | NN PP | PRP | TO VP
PP → IN NNP | IN NN
VB → want | Intent
DT → a | the | this | that
NN → book | Type
NNP → Suzy | City
IN → with | to | in | on  TO → to
Intent → book | cancel | find
City → Edmonton | Calgary | Rio
Type → flight | reservation | trip

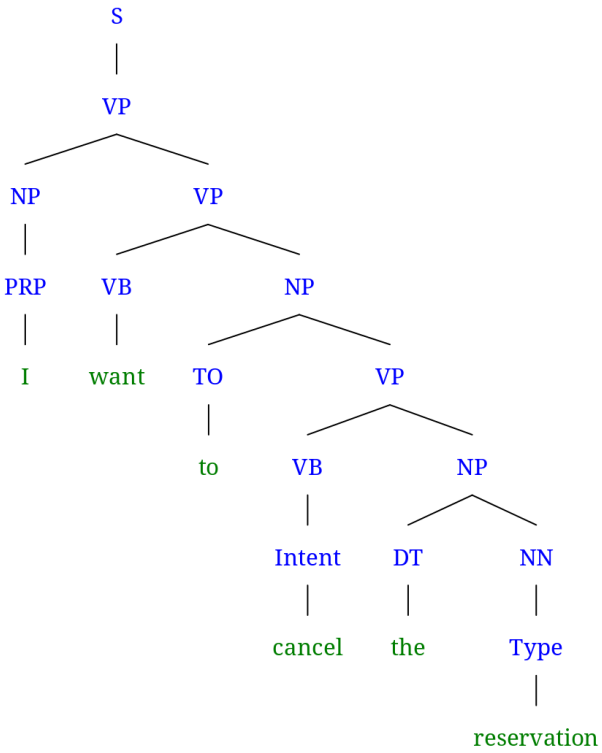





## Travel agent assistant

Specialized "semantic" grammar:

S → VP | NP VP  VP → VB NP
NP → DT NP | NN PP | PRP | TO VP
PP → IN NNP | IN NN
VB → want | Intent
DT → a | the | this | that
NN → book | Type
NNP → Suzy | City
IN → with | to | in | on  TO → to
Intent → book | cancel | find
City → Edmonton | Calgary | Rio
Type → flight | reservation | trip





## Relationship to other modules

1. Ambiguity
2. Parts-of-Speech
3. Parsing
    - Probabilistic CFGs (to help with ambiguous parses).
4. Language models and understanding
5. Intent classification, slot filling





## What you should know or be able to do

- Create a grammar from a tagged corpus
- Convert a CFG into CNF (parsing module)
- Generate grammatical sentences using CFGs and PCFGs
- Explain what are constituents
- Explain sentence-level ambiguity and identify ambiguous sentences
- Compute the probability of a sentence given a PCFG
- Explain how PCFGs solve ambiguity